

Amazon-Style E-Commerce Clone — Starter Project

A beginner-friendly full-stack starter project for learning purposes. It uses a React frontend, Node.js/Express backend, and MongoDB. It is not Amazon's proprietary source code, branding, or production system.

Project structure

```
amazon-clone/  
  client/  
    src/  
      App.jsx  
      main.jsx  
      index.css  
      package.json  
    server/  
      server.js  
      package.json  
      .env  
  README.md
```

client/package.json

```
{
  "scripts": {
    "dev": "vite"
  },
  "dependencies": {
    "axios": "^1.7.9",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^6.0.7"
  }
}
```

client/src/main.jsx

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <React.StrictMode><App /></React.StrictMode>
);
```

client/src/App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";

const API = "http://localhost:5000/api/products";

export default function App() {
  const [products, setProducts] = useState([]);
  const [search, setSearch] = useState("");
  const [cart, setCart] = useState([]);

  useEffect(() => {
    axios.get(API).then(res => setProducts(res.data));
  }, []);

  const filtered = products.filter(p =>
    p.name.toLowerCase().includes(search.toLowerCase())
  );

  const addToCart = product => setCart([...cart, product]);

  return (
    <div>
      <header className="topbar">
        <div className="logo">shop<span>clone</span></div>
        <input
          value={search}
          onChange={e => setSearch(e.target.value)}
          placeholder="Search products"
        />
        <div className="cart">■ Cart ({cart.length})</div>
      </header>

      <section className="hero">
        <h1>Online Shopping</h1>
        <p>Find products at great prices.</p>
      </section>

      <main className="grid">
        {filtered.map(product => (
          <article className="card" key={product._id}>
            <img src={product.image} alt={product.name} />
            <h2>{product.name}</h2>
            <p className="price">■{product.price}</p>
            <button onClick={() => addToCart(product)}>Add to Cart</button>
          </article>
        ))}
      </main>
    </div>
  );
}
```

```
);  
}
```

client/src/index.css

```
* { box-sizing: border-box; }  
body { margin: 0; font-family: Arial, sans-serif; background: #eaeaed; color: #222; }  
.topbar {  
  display: flex; align-items: center; gap: 20px; padding: 12px 24px;  
  background: #131921; color: white; position: sticky; top: 0; z-index: 2;  
}  
.logo { font-size: 24px; font-weight: bold; min-width: 130px; }  
.logo span { color: #ff9900; }  
.topbar input { flex: 1; max-width: 800px; padding: 12px; border: 0; border-radius: 4px; }  
.cart { white-space: nowrap; }  
.hero { background: white; padding: 40px 8%; margin-bottom: 20px; }  
.grid {  
  display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 18px; padding: 20px;  
}  
.card { background: white; padding: 16px; border-radius: 5px; }  
.card img { width: 100%; height: 200px; object-fit: contain; }  
.card h2 { font-size: 17px; }  
.price { font-size: 22px; font-weight: bold; }  
button {  
  width: 100%; padding: 10px; border: 0; border-radius: 20px;  
  background: #ffd814; cursor: pointer;  
}  
button:hover { background: #f7ca00; }
```

server/package.json

```
{  
  "scripts": { "start": "node server.js", "dev": "node --watch server.js" },  
  "dependencies": {  
    "cors": "^2.8.5",  
    "dotenv": "^16.4.7",  
    "express": "^4.21.2",  
    "mongoose": "^8.9.5"  
  }  
}
```

server/server.js

```
require("dotenv").config();  
const express = require("express");  
const cors = require("cors");  
const mongoose = require("mongoose");  
  
const app = express();  
app.use(cors());  
app.use(express.json());  
  
const Product = mongoose.model("Product", new mongoose.Schema({  
  name: String,  
  price: Number,  
  image: String  
}));  
  
app.get("/api/products", async (req, res) => {  
  try {  
    const products = await Product.find().limit(50);  
    res.json(products);  
  } catch (err) {  
    res.status(500).json({ error: "Unable to load products" });  
  }  
});  
  
app.post("/api/products", async (req, res) => {  
  try {  
    const product = await Product.create(req.body);  
    res.status(201).json(product);  
  } catch (err) {  
    res.status(400).json({ error: "Invalid product" });  
  }  
});  
mongoose.connect(process.env.MONGO_URI)
```

```
.then(() => app.listen(5000, () =>
  console.log("API running on http://localhost:5000")
))
.catch(err => console.error("MongoDB connection failed:", err));
```

server/.env

MONGO_URI=mongodb://127.0.0.1:27017/amazon_clone

README.md

Amazon-Style E-Commerce Clone

Requirements

- Node.js 18+
- MongoDB
- VS Code or another code editor

1. Start the backend

```
cd server
npm install
npm run start
```

2. Start the frontend

```
cd client
npm install
npm run dev
```

Open the Vite URL shown in the terminal, normally:

<http://localhost:5173>

3. Add sample products

Use POST <http://localhost:5000/api/products> with JSON such as:

```
{
  "name": "Wireless Headphones",
  "price": 1499,
  "image": "https://placeholder.co/600x400?text=Headphones"
}
```

Features in this starter

- Product listing
- Product search
- Add to cart counter
- REST API
- MongoDB storage
- Responsive basic layout

For a larger project, add authentication, product details, categories, checkout, orders, payment gateway integration, admin dashboard, validation, image upload, and security controls. Do not use Amazon trademarks, logos, copyrighted assets, or private APIs in your clone.

How to run

1. Create the folder structure shown above and save each code section in its matching file.
2. Install Node.js and MongoDB. Start MongoDB locally.
3. Open two terminals. In the first, run **cd server**, **npm install**, then **npm run start**.
4. In the second, run **cd client**, **npm install**, then **npm run dev**.
5. Open the local Vite address displayed in the terminal.

Important

This is an educational Amazon-style clone. It does not copy Amazon's proprietary application code. For a real deployment, add authentication, authorization, secure secrets, input validation, database indexes, rate limiting, payment-provider integration, HTTPS, logging, and proper error handling.